1 How to use my program

My program is executed from the terminal. The syntax to do so is as follows:

```
stack run -- (-f|--file|--filename FILENAME) [-a|--ambiguous]
```

Note that the `--` after `stack run` is necessary for the arguments to be passed to my program instead of to Stack.

More detail on the functions of the program is given below. You can also run

```
stack run -- --help
```

to view details on all possible options, alongside any aliases.

1.1 Parsing `.gridlock` files strictly

The primary way to use this program is to parse `.gridlock` files strictly. This means that the files must directly conform to the standards laid out in the coursework specification. To do so, use the following command:

```
stack run -- --file=<FILENAME>
```

Where `<FILENAME>` is the name of the `.gridlock` file to be parsed. If the file to be parsed is not in the working directory, this can instead be a relative path to the files location. The name of the specified file must end in `.gridlock` and the file itself must fully conform to the standard laid out in the coursework specification for the parse to work. Note that the player's names must only contain alphanumeric characters, so no symbols or spaces.

Executing this command will read the `.gridlock` file and produce a human-readable output displaying the game described in the file. This includes verifying that the game does actually end when the file says it does (e.g. that there are no further moves that could be made).

1.2 Parsing Gridlock files non-strictly, or with ambiguity

This program can also handle files describing Gridlock games that do not conform fully to the standard laid out in the coursework specification. These files introduce room for ambiguity and so parsing them required making some assumptions, listed below. To use the program in this mode, add the `-a` flag (standing for "ambiguity"), like so:

```
stack run -- --file=<FILENAME> -a
```

Once again, `<FILENAME>` must be the name (or path) of the file to parse, but it needn't end in `.gridlock` when parsing non-strictly (although it is recommended to still store Gridlock files with the `.gridlock` file extension for clarity).

This mode is able to correctly parse all standard `.gridlock` files that the strict mode can handle. In addition:

- The "`GRIDLOCK`" line at the top of the file is not required.
- The "`Player:`" line can be incomplete (only listing a single player), or omitted entirely. In these cases, the names of the players will be taken from the first two moves. If there are less than two moves, then the names "Player 1" and "Player 2" will be used to name any remaining players. Hence, this makes the assumption that the first two moves of the game are correct, with the correct players playing in the correct order.
- The "`Grid:`" line is required, but arbitrary horizontal white-space is allowed between the colon, "`x`" and numbers. For example,

  ```
  Grid:3    x  3.
  ```

  is valid.
- The "`Colours:`" line can be incomplete (only listing some of the colours in the game), or omitted entirely. When a colour not listed in this line is encountered, it is added to the set of colours used in the game (which is important for checking that the game ended correctly). Hence, this makes the assumption that all moves in the game use allowed colours, and that there are no allowed colours that weren't used anywhere.
- The "`[player name] cannot move.`" and "`[player name] wins!`" lines can be omitted. If they are included, they must be correct, and the program still confirms that the game ended correctly when they are omitted.
- Newlines are not required (other than after the "`GRIDLOCK`" line, if it is included), and in fact any arbitrary white-space is allowed between "phrases". For example,

  ```
  Amy plays red at (0,0). Bill plays red at (2,0).



  Amy plays green at (1,1).          Bill plays red at (0,2).
  Amy plays red at (2,2).
  ```

  is valid.
- Full stops are not required, although newlines are then required after any line where they are omitted.

For example, the following could be parsed in this mode:

```
    Players: Amy. Grid:3    x  3


Amy plays red at (0,0).
Bill plays red at (2,0).


Amy plays green at (1,1).
                  Bill plays red at (0,2).
Amy plays red at (2,2).

Amy wins!
```

This parses the same as the following strict file:

```
GRIDLOCK
Players: Amy, Bill.
Grid: 3x3.
Colours: red, green.

Amy plays red at (0,0).
Bill plays red at (2,0).
Amy plays green at (1,1).
Bill plays red at (0,2).
Amy plays red at (2,2).
Bill cannot move.
Amy wins!
```

2 Program architecture

I decided to keep the program as a simple command line application, with no TUI, as this felt excessive for my needs. I used the optparse-applicative library to implement this instead of anything fancier like Brick.

This program is split across various files. The main entry point is `main.hs` in the `app` folder. In addition, there are the following files in the `src` folder:

- `Args.hs`
- `ColourSquares.hs` (unchanged from skeleton code)
- `DrawGrid.hs`

- `Parser.hs`
- `Types.hs` (unchanged from skeleton code)
- `Utils.hs`

I will now go through these in turn in an order that makes sense to me.

2.1 `Types.hs`

`Types.hs` was already written and I have left it untouched. It defines the types used to represent games of Gridlock.

2.2 `ColourSquares.hs`

`ColourSquares.hs` was already written and I have left it untouched. It provides a function `square` which takes a `Colour` and returns a `String` containing a square of that colour which can be displayed to the terminal. This is used in `DrawGrid.hs` only.

2.3 `DrawGrid.hs`

`DrawGrid.hs` contains a single function, `drawGrid`, which takes a `Grid` and returns a human-readable representation of it as a `String`.

2.4 `Utils.hs`

`Utils.hs` is a small file containing some definitions that didn't fit well elsewhere. These are all explained in comments in the file.

2.5 `Parser.hs`

`Parser.hs` contains the code for parsing `.gridlock` files into `GameRecord`s. Most of my development time was spent in this file, primarily working on making the code as nice as possible using monad transformers. The stack I ended up with is:

```
type MonadStack = ReaderT Bool (
                  ParsecT Void String (
                  State GameRecord
                  ))
```

The most important monad/monad transformer here is `ParsecT`, from the Megaparsec library, which handles the actual parsing.

The next most important is `State` from the MTL library. Many stages of parsing that occur here rely on data already parsed previously. Moreover, the code putting together the final `GameRecord` became messy. The `State` monad simplifies this by allowing a stateful `GameRecord` to be threaded

through the parsers. It can be added to as soon as data has been parsed, made easier using the `addGrid`, `addPlayer` and `addColour` functions which add their respective type of data to the stateful `GameRecord`, and then this already parsed data can be read as necessary immediately. This is much neater than manually providing each bit of data to the functions that need it, and also allows the lines describing moves to be parsed more simply.

Finally, the `ReaderT` monad transformer, also from the MTL library, provides read-only access to a Boolean value indicating whether ambiguous files should be parsed (i.e. non-strict parsing) when `True`, or not parsed (i.e. strict parsing) when `False`. I introduced this after realising that I was manually providing this value to almost every parsing function.

The main function in this file is `parseGame` which is responsible for calling on the other functions to parse the entire game. `parsePlayers`, `parseSize` and `parseColours` all parse the information given at the top of the file. `parseMove` is responsible for parsing the actual moves, and data is transferred between the individual occurrences of this function (which is ran an arbitrary number of times, until the "[player] cannot move.", "[player] wins!" lines are reached, or the file being parsed ends) through the `State` monad.

2.6 `Args.hs`

`Args.hs` is responsible for handling the command line arguments passed to the program at runtime using the optparse-applicative library. This library provides a simple way to parse and handle these arguments using applicatives (these are used instead of monads in order to allow arguments to be provided in any order). I used this library as I thought it looked simple to use and it saved me having to write my own code for this task which would have meant less time to develop the functionality of my program.

2.7 `Main.hs`

`Main.hs` contains the main entry point to the program, the `main` function. This is the only function in the program to use the `IO` monad, keeping every other function in the program pure. It reads and parses the runtime arguments, reads the specified file, runs the parser and outputs the game in a nice way.

3 Experience writing the program

I was originally planning to add far more features than I have to the program. For example, I wanted to include the ability to "repair" files that can be parsed non-strictly but not strictly into fully-conforming `.gridlock` files. Moreover, I wanted to include the ability to play games of Gridlock against an opponent, and possibly also an "AI" opponent, and save these games to `.gridlock` files. However, due to time constraints and it taking longer than planned to implement the functionality I have implemented to a standard I'm happy with, I have had to scrap these features.

The bulk of my development time was spent on the parsers in `Parser.hs`. I got a functioning version of this file together quite quickly once I started but I then spent a lot of time refactoring. Specifically, I spent a lot of time on the monad stack, making sure I was fully taking advantage of monad transformers. I was planning to make the "`Grid:`" line optional in the non-strict mode as

well. To do this, I was planning to add the `Tardis` monad to the stack: as the program went through each line describing a move, it would ensure that the value sent to the future by the end of this process was the largest x and y coordinate of any move. This would then be sent back in time to near the start of the function and used as the size of the grids in the game. This would then allow the empty grids to be created and their `rep`s populated with all of the required empty `Cell`s. However, while I was able to get this to compile, it looped infinitely and I couldn't see an easy way around this without having to go through the Gridlock file (or some other representation of the game) more than once which I decided, somewhat arbitrarily, that I didn't want to do. However, I did find this failed attempt extremely useful in bettering my understanding of monad transformers.

I considered making use of the non-empty library for the `Grids` and possibly `Players` and `Colours` (making use of the `Optional` and `Empty` constructors) fields of `GameRecord` type. However, this created difficulties with initialising the `State` monad and would have required significantly re-working how I made use of that monad which I didn't want to do at the stage of development that I decided to explore this. Moreover, the `T` type (the main type provided by the non-empty library) doesn't have a `read` instance which is needed for the automated tests in the skeleton code, meaning I would have to write one myself for those to work.

4 Resources

Other than the lecture notes, I made extensive use of Hackage. Pages I used primarily include:

- `https://hackage.haskell.org/package/optparse-applicative`
- `https://hackage.haskell.org/package/megaparsec-9.7.0/docs/Text-Megaparsec.html`
- `https://hackage.haskell.org/package/megaparsec-9.7.0/docs/Text-Megaparsec-Char.html`
- `https://hackage.haskell.org/package/megaparsec-9.7.0/docs/Text-Megaparsec-Char-Lexer.html`
- `https://hackage.haskell.org/package/mtl-2.3.1/docs/Control-Monad-State-Lazy.html`
- `https://hackage.haskell.org/package/mtl-2.3.1/docs/Control-Monad-Reader.html`
- `https://hackage.haskell.org/package/containers-0.7/docs/Data-Map-Lazy.html`
- `https://hackage.haskell.org/package/containers-0.8/docs/Data-Set.html`
- `https://hackage.haskell.org/package/base-4.21.0.0/docs/System-IO.html`
- `https://hackage.haskell.org/package/base-4.14.1.0/docs/Control-Applicative.html#t:Alternative`
- `https://hackage.haskell.org/package/parser-combinators-1.3.0/docs/Control-Monad-Combinators.html`

- https://hackage.haskell.org/package/base-4.21.0.0/docs/Control-Monad.html#v:when

Other webpages I used are:

- https://en.wikibooks.org/wiki/Haskell/Monad_transformers
- https://wiki.haskell.org/All_About_Monads
- https://wiki.haskell.org/Monad_Transformers_Explained
- https://blog.ocharles.org.uk/posts/2014-12-04-record-wildcards.html
- https://ghc.gitlab.haskell.org/ghc/doc/users_guide/exts/record_wildcards.html
- https://ghc.gitlab.haskell.org/ghc/doc/users_guide/exts/record_puns.html

Pages I looked at in the process of my completing this coursework but didn't (consciously) use any ideas from include:

- https://markkarpov.com/tutorial/megaparsec.html#parse-errors
- https://hackage.haskell.org/package/megaparsec-9.7.0/docs/Text-Megaparsec-Error.html
- https://hackage.haskell.org/package/tardis-0.5.0/docs/Control-Monad-Tardis.html
- https://hackage.haskell.org/package/tardis-0.5.0/docs/Control-Monad-Tardis-Class.html
- https://hackage.haskell.org/package/non-empty-0.3.5/docs/Data-NonEmpty.html
- https://hackage.haskell.org/package/base-4.21.0.0/docs/Data-STRef.html
- https://hackage.haskell.org/package/filepath-1.5.4.0/docs/System-FilePath.html
- https://hackage.haskell.org/package/brick
- https://rebeccaskinner.net/posts/2021-08-25-introduction-to-type-level-programming.html
- https://ghc.gitlab.haskell.org/ghc/doc/users_guide/exts/recursive_do.html